Contents

- Java 8+
- Design
- Spring Boot
- Database

Java interview guide, part 2: APIs, design, frameworks and data

This is a guide to the interview, not just answers. Each question opens with what the interviewer is really testing and closes with the follow-ups they will ask next. Coworker-coaching voice throughout. This is **Part 2 of 3**.

Java 8+

32. Explain lambda internals.

What they are testing: whether you know a lambda is not just syntactic sugar for an anonymous class, a common misconception.

Answer. A lambda is **not** compiled to an anonymous inner class. The compiler turns it into a **private method** holding the body, plus an `invokedynamic` call site that, on first execution, uses `LambdaMetafactory` to spin up the implementing object. So there is **no extra `.class` file per lambda** (unlike anonymous classes), the instance can be reused, and `this` refers to the enclosing instance, not a new object. The practical payoff is lighter footprint and a small one-time bootstrap cost on first call.

Follow-ups. *Why not just use anonymous classes under the hood?* That would generate a class per lambda, bloating the jar and class-loading; `invokedynamic` defers the strategy to runtime. *Does a lambda capture variables?* Only effectively-final locals, by value.

33. What is invokedynamic?

What they are testing: depth on the JVM instruction behind lambdas (and dynamic languages); keep it tight.

Answer. `invokedynamic` is a **bytecode instruction that leaves the target method unbound until the first time it runs**, when a "bootstrap method" decides what to actually call and links it. It was added for dynamic languages on the JVM, and lambdas reuse it: the bootstrap (`LambdaMetafactory`) builds the lambda implementation lazily. So it is the mechanism that lets the JVM decide *at runtime* how to create the lambda, rather than baking it in at compile time.

34. Stream v/s Collection.

What they are testing: whether you understand a stream is a **pipeline**, not a data structure.

Answer. A `Collection` **stores** data; a `Stream` **describes a computation over** data and holds nothing itself. Key differences: a collection is **eagerly** populated and you can iterate it many times; a stream is **lazy** and **single-use** (once consumed, it is done). A collection is about storage and access; a stream is about **declarative processing** (filter/map/reduce), and it can go parallel with `.parallelStream()` . So they are complementary, you have a collection, you *stream* it to process, then collect back.

Follow-ups. *Can you reuse a stream?* No, it throws `IllegalStateException` on a second terminal op. *Does a stream modify the source?* No, it produces a new result.

35. Why are streams lazy?

What they are testing: whether you understand that laziness enables optimisation, not just "it delays work."

Answer. Intermediate operations (`filter` , `map`) **do nothing until a terminal operation** (`collect` , `forEach`) runs. Laziness buys two things: **short-circuiting** (`findFirst` or `limit` can stop early without processing the whole source) and **loop fusion** (the pipeline runs in a single pass per element rather than one full pass per operation). So `list.stream().filter(...).map(...).findFirst()` does not filter the entire list, it pulls elements one at a time until the first match. That efficiency is the point of laziness.

Follow-ups. *What triggers execution?* The terminal operation. *Give a short-circuit example?* `anyMatch` , `findFirst` , `limit` all stop early.

36. Parallel stream, when should you avoid it?

What they are testing: judgment. Parallel streams look free but are a trap, and seniors know when *not* to use them.

Answer. Avoid a parallel stream when the work is **small** (the fork/join overhead dwarfs the gain), when the operations are **I/O-bound** (threads just block, and it uses the shared common `ForkJoinPool` , so one slow parallel stream can starve others), when there are **side effects or ordering dependencies** (parallelism reorders and races), or when the source **splits poorly** (a `LinkedList` does not partition well; arrays and `ArrayList` do). Rule of thumb: only reach for it with a **large dataset, CPU-bound, stateless, independent** operations, and even then, measure. The default should be a sequential stream.

Follow-ups. *Which pool does it use?* The shared `ForkJoinPool.commonPool` , which is why a blocking parallel stream can hurt the whole app. *When is it genuinely worth it?* Large in-memory CPU-heavy transforms, verified by benchmarking.

37. Explain Optional.

What they are testing: whether you use it correctly (return types, not everywhere), not just "it avoids null."

Answer. `Optional<T>` is a container that **explicitly represents "value or no value"**, so absence is in the type signature instead of a silent null. It reduces NPEs by **forcing the caller to handle the empty case** (`map` , `orElse` , `orElseThrow` , `ifPresent`). The senior nuance: it is designed for **return types**, not for fields or method parameters, and a bare `.get()` without checking just recreates the NPE, so use `orElse` / `orElseThrow` .

Follow-ups. *`Optional` as a field?* Discouraged, it is not `Serializable` and adds overhead; use null internally, `Optional` at the boundary. *`orElse` v/s `orElseGet` ?* `orElse` always evaluates its argument; `orElseGet` takes a supplier and only runs it when empty (use it when the default is expensive).

38. Difference between map() and flatMap().

What they are testing: a precise understanding, and this one **genuinely needs a code example** because the shapes are the whole point.

Answer. `map` transforms **each element into exactly one element**; `flatMap` transforms **each element into a stream and then flattens** all those streams into one. So `map` keeps the nesting, `flatMap` removes a level of it.

```java
// map: 1 element -> 1 element (keeps nesting)
List<List<String>> nested = ...;
nested.stream().map(List::size);          // Stream<Integer>  (fine, 1:1)
nested.stream().map(List::stream);        // Stream<Stream<String>>  (nested, usually not
        what you want)

// flatMap: 1 element -> a stream, then flattened
nested.stream()
      .flatMap(List::stream)              // Stream<String>  (one flat stream)
      .collect(Collectors.toList());
```

So you use `flatMap` whenever mapping produces collections/streams/`Optional`s and you want them merged into one flat stream.

Follow-ups. *When would you use flatMap?* Flattening nested collections, or chaining `Optional`s (`Optional.flatMap`). *map on Optional?* Same idea, `flatMap` avoids `Optional<Optional<T>>`.

39. Explain CompletableFuture.

What they are testing: whether you can do **composable async** work, a step up from raw threads and `Future`.

Answer. `CompletableFuture` represents an **async computation you can chain and combine without blocking**. You attach callbacks: `thenApply` (transform the result), `thenCompose` (chain another async call), `thenCombine` (join two futures), and handle failures with `exceptionally` or `handle`, all without calling a blocking `get()`. So instead of "start a thread, block on the result", you build a **pipeline of async steps** that runs as results arrive. It also lets you complete it manually (`complete()`) and pick the executor to run on.

Follow-ups. *`thenApply` v/s `thenCompose`?* `thenApply` maps to a plain value; `thenCompose` flattens when the next step itself returns a `CompletableFuture` (the async `flatMap`). *Default executor?* The common `ForkJoinPool` unless you pass your own, worth passing your own for blocking work.

40. Future v/s CompletableFuture.

What they are testing: why `CompletableFuture` (Java 8) replaced most uses of `Future`.

Answer. `Future` (Java 5) can only be **blocked on** (`get()`) or polled; you cannot chain it, combine it, or react to completion. `CompletableFuture` fixes all of that.

	Future	**CompletableFuture**
Get result	Blocking `get()` only	Non-blocking callbacks (`thenApply`, ...)
Compose / combine	No	Yes (`thenCompose`, `thenCombine`)
Manual completion	No	Yes (`complete()`)
Exception handling	Clunky (try/catch on `get`)	`exceptionally` / `handle`

So `Future` is a read-only handle you block on; `CompletableFuture` is a composable async building block. Use `CompletableFuture` for anything non-trivial.

Design

41. Explain the SOLID principles.

What they are testing: whether you can state all five crisply and, ideally, give a one-line example. They may then drill into one.

Answer. Five principles for changeable OO code:

	Principle	Idea
S	Single Responsibility	A class should have one reason to change
O	Open/Closed	Open for extension, closed for modification
L	Liskov Substitution	A subtype must work anywhere its base type is expected
I	Interface Segregation	Many small interfaces beat one fat one
D	Dependency Inversion	Depend on abstractions, not concretions

The line that lands: the common thread is **managing change**, each principle reduces how far a change ripples.

Follow-ups. *Give an OCP example?* Add a new payment type as a new class implementing an interface, rather than editing a `switch`. *Classic LSP violation?* `Square extends Rectangle` breaks the independent width/height contract. *DIP v/s Dependency Injection?* DIP is the principle (depend on abstractions), DI is a technique to supply them.

42. Explain the Factory pattern.

What they are testing: whether you know *why* to use it (decoupling from concrete classes), not just the mechanics.

Answer. A factory **moves object creation behind a method** so callers ask for an abstraction instead of doing `new ConcreteClass()`. That **decouples** the caller from the concrete type, so you can swap implementations, add new ones, and mock in tests without touching call sites. The variants: **Simple Factory** (one method picks the type, an idiom, not GoF), **Factory Method** (subclasses decide the concrete product, supports OCP), and **Abstract Factory** (creates whole families of related products). Use it when the concrete type varies or creation is non-trivial; a plain constructor is fine otherwise.

Follow-ups. *Factory Method v/s Abstract Factory?* Factory Method makes one product via inheritance; Abstract Factory makes a family via composition. *Does it help testing?* Yes, callers depend on the abstraction, so you inject a fake.

43. Singleton implementations, and which do you recommend?

What they are testing: whether you know the thread-safety trade-offs across implementations and have a defensible recommendation.

Answer. The main ones: **eager** (simple, but built even if unused), **synchronized method** (thread-safe but locks every call), **double-checked locking** (locks only on first creation, needs `volatile`), **Bill Pugh holder idiom** (lazy and thread-safe via class-loading, no locking), and **enum** (thread-safe, plus immune to reflection and serialization attacks).

I **recommend Bill Pugh** for a plain class singleton (lazy, thread-safe, no volatile, simplest correct option), or an **enum** when I want it bulletproof against reflection and serialization. I avoid the synchronized-method version (needless locking) and hand-rolled DCL (easy to get wrong).

Follow-ups. *Why not DCL?* It works but is fiddly (the volatile is easy to forget); Bill Pugh gives the same benefit without the traps. *Downside of enum?* Eager, and cannot extend a class. *Bigger picture?* In a Spring app, let the container manage it as a singleton bean instead.

44. Explain the Strategy pattern.

What they are testing: whether you recognise it as "encapsulate interchangeable behaviour", and that lambdas often replace it now.

Answer. Strategy **defines a family of interchangeable algorithms behind a common interface**, so you can swap the behaviour at runtime without changing the caller. The classic example is a `PaymentStrategy` interface with `CreditCard`, `PayPal`, `Crypto` implementations, and the checkout picks one. It is really OCP in action, add a new strategy without touching existing code. In modern Java a strategy interface is often just a **functional interface you pass a lambda to**, so `Comparator` in a `sort` is a strategy.

Follow-ups. *Strategy v/s just an if/else?* The strategy lets you add behaviours without editing the chooser, and inject them for testing. *Relation to lambdas?* A single-method strategy is a functional interface; lambdas remove the boilerplate.

45. Explain the Builder pattern.

What they are testing: when to use it (many optional/complex parameters), and recognising the "telescoping constructor" problem it solves.

Answer. The Builder **constructs a complex object step by step through a fluent API**, then produces it with `build()`. It solves the **telescoping constructor** problem (constructors with many parameters, where you cannot tell what `new Pizza(12, true, false, true)` means) and makes it easy to build **immutable** objects with optional fields.

```
Pizza p = new Pizza.Builder()
    .size(12)
    .topping("cheese")
    .topping("mushroom")
    .build();     // readable, immutable, no telescoping constructor
```

Follow-ups. *Builder v/s constructor?* Use a builder when there are many (especially optional) parameters or you want immutability; a constructor is fine for a few required fields. *Where have you seen it?* `StringBuilder`, `Stream.Builder`, Lombok's `@Builder`.

46. Explain the Observer pattern.

What they are testing: the publish-subscribe idea and where it shows up; keep it concise.

Answer. Observer lets a **subject notify a list of observers automatically when its state changes**, so the observers react without the subject knowing their concrete types. It is the basis of **event-driven** code: UI listeners, and at larger scale, event buses and pub/sub messaging. It supports OCP, you add a new observer without changing the subject. The watch-out is memory leaks from **observers that never unsubscribe**, and cascading notifications.

Follow-ups. *Real examples?* GUI event listeners, Spring's `ApplicationEvent`, message-queue consumers. *Downside?* Unsubscribing is easy to forget (leak), and notification order is not guaranteed.

47. Composition v/s Inheritance.

What they are testing: whether you know the modern guidance (favour composition) and *why*, a senior design staple.

Answer. Inheritance is an **"is-a"** relationship (a `Car` is a `Vehicle`); composition is a **"has-a"** (a `Car` has an `Engine`). The guidance is **favour composition over inheritance**, because inheritance is **tight coupling**: a subclass depends on its parent's internals, a change to the base can break subclasses (the "fragile base class" problem), and Java's **single inheritance** means you spend your one slot. Composition is more flexible, you build behaviour by combining objects, swap parts at runtime, and it composes without limit. Use inheritance only for a genuine, stable "is-a" with real shared behaviour; reach for composition otherwise.

Follow-ups. *When is inheritance still right?* A true is-a with a stable base and real shared implementation (a template-method flow). *How does this relate to Strategy?* Strategy is composition, you inject behaviour instead of subclassing to override it.

Spring Boot

48. Explain the Spring bean lifecycle.

What they are testing: whether you know the container manages a bean's whole life, and the hook points, useful for resource setup/teardown.

Answer. The container drives it: **instantiate** the bean, **inject dependencies**, run **aware** callbacks (like `BeanNameAware`), then `BeanPostProcessor` **before-init**, then **init** (`@PostConstruct`, then `InitializingBean.afterPropertiesSet`, then a custom init method), then `BeanPostProcessor` **after-init**, at which point the bean is **ready**. On shutdown it runs **destroy** (`@PreDestroy`, then `DisposableBean.destroy`). The practical hooks you use are `@PostConstruct` (open a resource, warm a cache once dependencies are wired) and `@PreDestroy` (clean up).

Follow-ups. *Where does AOP (like `@Transactional`) get applied?* In a `BeanPostProcessor`, it wraps the bean in a proxy after init. *When does the singleton get created?* Eagerly at startup by default (prototypes are created on demand).

49. What are the Spring bean scopes?

What they are testing: knowing that singleton is the default and what the web scopes are; a quick check.

Answer.

Scope	Lifetime
singleton	**Default**, one instance per container
prototype	A new instance every time it is requested
request	One per HTTP request (web)
session	One per HTTP session (web)
application	One per servlet context (web)
websocket	One per WebSocket session

The one that catches people: **singleton is per Spring container, not per JVM**, and a singleton bean must be **stateless** (it is shared across all threads).

Follow-ups. *Prototype in a singleton?* The prototype is injected once at creation, so it does not get "re-created" per use, use a provider/lookup if you need a fresh one each call.

50. Spring singleton bean v/s Java singleton.

What they are testing: whether you know these are different concepts (a favourite trap).

Answer. A **Java singleton** enforces **one instance per JVM** with a private constructor and static access, self-managed global state. A **Spring singleton bean** is **one instance per container, per bean name**, managed by the framework and handed to you via **dependency injection**, no private constructor, no static access, and you can swap it in tests. So Spring gives you the "one shared instance" benefit **without** the testability and coupling problems of the classic pattern. Different scope (JVM v/s container) and different mechanism (static v/s DI).

Follow-ups. *Two Spring contexts?* You get two instances, one per container, which surprises people expecting JVM-wide uniqueness.

51. Constructor injection v/s field injection.

What they are testing: whether you know constructor injection is the recommended default, and why. This warrants a short snippet.

Answer. Prefer constructor injection. It lets fields be `final` (immutable, guaranteed set), makes dependencies **explicit and testable** (you just `new` the class with mocks, no reflection), and **fails fast** on a missing dependency and on **circular dependencies** at startup. **Field injection** (`@Autowired` on a field) is convenient but hides dependencies, cannot be `final`, and needs reflection to test.

```
@Service
class OrderService {
    private final PaymentGateway gateway;      // final: immutable, always set
    OrderService(PaymentGateway gateway) {     // constructor injection
        this.gateway = gateway;
    }
    // test: new OrderService(mockGateway) — no Spring, no reflection needed
}
```

Follow-ups. *Why does field injection make testing harder?* No constructor to pass mocks in, you need reflection or the container. *Circular dependency?* Constructor injection detects it at startup (fails loudly); field injection may hide it.

52. Explain Spring Boot auto-configuration.

What they are testing: whether you understand the "convention over configuration" magic, not treat it as a black box.

Answer. Auto-configuration **configures beans automatically based on what is on the classpath and what you have not already defined**. `@SpringBootApplication` enables it, and at startup Spring reads a list of auto-configuration classes (from `META-INF/spring/...AutoConfiguration.imports`), each guarded by `@Conditional` annotations: `@ConditionalOnClass` (this library is present), `@ConditionalOnMissingBean` (you have not defined your own), and so on. So if H2 is on the classpath and you defined no `DataSource`, it configures one; if you define your own, it backs off. That is how Spring Boot "just works" while still letting you override anything.

Follow-ups. *How do you override it?* Define your own bean, `@ConditionalOnMissingBean` makes the auto-config step back off. *How do you debug it?* `--debug` prints the auto-configuration report (what matched and why).

53. What happens when a Spring Boot application starts?

What they are testing: an end-to-end mental model of startup.

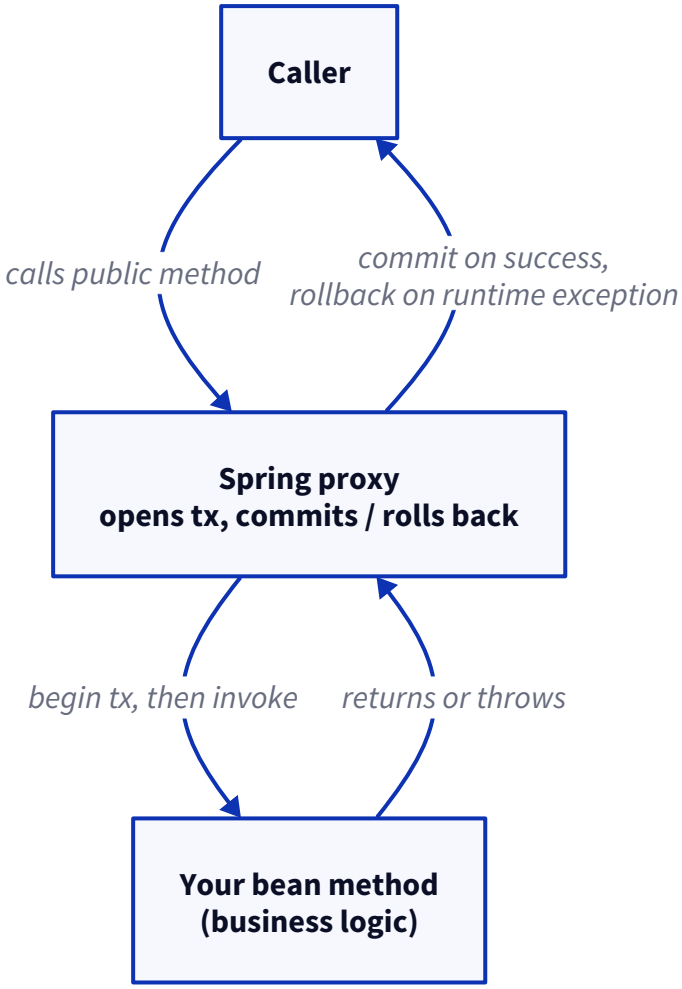
Answer. `SpringApplication.run()` kicks off: it **creates the application context**, **scans for components** (`@Component`, `@Service`, `@Controller`, and so on), applies **auto-configuration**, then **instantiates and wires the singleton beans** (running their lifecycle callbacks), starts the **embedded server** (Tomcat by default) for a web app, and finally publishes an **application-ready event**. At that point it is serving requests. So: context, scan, auto-configure, build beans, start server, ready.

Follow-ups. *When are `@PostConstruct` hooks run?* During bean initialisation, before the app is marked ready. *What is the embedded server?* Tomcat by default (Jetty/Undertow optional), which is why a Boot app is a runnable jar, not a war you deploy.

54. How does @Transactional work?

What they are testing: whether you know it is **proxy-based AOP**, which explains all its gotchas.

Answer. Spring wraps your bean in a **proxy**. When you call a `@Transactional` method **through the proxy**, the proxy **opens a transaction before** the method and **commits after** it returns, or **rolls back** if it throws. So the transaction management is not in your code, it is in the proxy around it.



The default rollback rule catches people: it **rolls back on unchecked (runtime) exceptions and `Error`, but NOT on checked exceptions** unless you configure `rollbackFor`. This proxy model is exactly why the next two questions exist.

Follow-ups. *Default rollback?* Runtime exceptions only; use `rollbackFor` for checked. *Where do you put it?* On the service layer, wrapping a full unit of work.

55. Explain transaction propagation levels.

What they are testing: whether you know what happens when one transactional method calls another.

Answer. Propagation decides how a method joins (or does not join) an existing transaction:

Propagation	Behaviour
REQUIRED	**Default**, join the current tx or start one
REQUIRES_NEW	Suspend the current tx, start a fresh independent one
NESTED	Run in a savepoint within the current tx (partial rollback)
SUPPORTS	Join if one exists, else run without a tx
MANDATORY	Must run in an existing tx, else throw
NOT_SUPPORTED	Suspend any tx, run non-transactionally
NEVER	Throw if a tx exists

The one worth knowing well is **REQUIRES_NEW**: use it when an inner operation (like an audit log) must **commit independently** even if the outer transaction rolls back.

Follow-ups. *REQUIRED v/s REQUIRES_NEW?* REQUIRED shares the outer tx (rolls back together); REQUIRES_NEW is independent. *Does REQUIRES_NEW help audit logging?* Yes, the log survives an outer rollback.

56. Explain transaction isolation levels.

What they are testing: whether you connect the Spring setting to the underlying database behaviour.

Answer. These are the **database isolation levels**, exposed on `@Transactional(isolation = ...)`. They trade consistency against concurrency by controlling which read anomalies are allowed, dirty reads, non-repeatable reads, and phantom reads. The full breakdown of what each level prevents is in the **Database section (Q58)**; in Spring you just pick one (or leave the database default, often READ_COMMITTED), and Spring passes it to the driver.

Follow-ups. *Default?* Usually the database default (READ_COMMITTED on Postgres/Oracle, REPEATABLE_READ on MySQL InnoDB). *When would you raise it?* When you need repeatable reads or to prevent phantoms, at the cost of more locking/contention.

57. Why doesn't @Transactional work on private methods?

What they are testing: whether you truly understand the proxy mechanism, this is the payoff question for Q54.

Answer. Because `@Transactional` works through a **proxy**, and a proxy can only intercept **public methods called from outside the bean**. Two cases break it: **private methods** (the proxy cannot override them, so it never sees the call), and **self-invocation** (`this.otherMethod()` from within the same bean, which calls the real object directly and bypasses the proxy entirely). So the annotation is silently ignored. The fix is to make the method **public and call it from another bean** (through the proxy), or restructure so the transactional entry point is the externally-called public method.

Follow-ups. *Self-invocation fix?* Move the transactional method to a separate bean, or inject a reference to the proxy. *Why public only?* Proxies (JDK dynamic or CGLIB) intercept public methods; private ones are invisible to them.

Database

58. Explain ACID.

What they are testing: the fundamentals of what a transaction guarantees.

Answer. ACID is the four guarantees a transaction gives:

- **Atomicity**: all-or-nothing, either every step commits or none does.
- **Consistency**: the transaction moves the database from one valid state to another (constraints hold).
- **Isolation**: concurrent transactions do not step on each other (as if run in sequence, tunable via isolation levels).
- **Durability**: once committed, it survives a crash (written to durable storage).

Follow-ups. *Which does isolation level tune?* The I, how much concurrent transactions can see of each other. *Where does durability come from?* Write-ahead logs / commit to disk.

59. Explain isolation levels.

What they are testing: whether you know the four levels and the exact anomaly each one prevents, a senior-level detail.

Answer. Isolation levels trade consistency against concurrency by controlling three read anomalies: a **dirty read** (reading another transaction's uncommitted change), a **non-repeatable read** (re-reading a row and getting a different value because someone committed a change), and a **phantom read** (re-running a query and getting new rows).

Level	Dirty read	Non-repeatable read	Phantom read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

Higher isolation means more consistency but **more locking and less concurrency**, so you pick the lowest level that is correct for the use case. READ COMMITTED is the common default.

Follow-ups. *SERIALIZABLE downside?* Most locking, worst throughput, and more deadlocks/serialization failures. *MySQL v/s Postgres default?* MySQL InnoDB defaults to REPEATABLE READ, Postgres to READ COMMITTED.

60. Optimistic v/s pessimistic locking.

What they are testing: whether you can pick the right concurrency-control strategy for a contention level.

Answer. Pessimistic locking assumes conflicts are likely, so it **locks the row up front** (`SELECT ... FOR UPDATE`) and others wait. **Optimistic locking** assumes conflicts are rare, so it **takes no lock**, and instead checks a **version column** at update time: if the version changed since you read it, the update fails and you retry. Use **optimistic** for low-contention, read-heavy workloads (better throughput, no locks held); use **pessimistic** for high-contention or when a retry is expensive or unacceptable.

Follow-ups. *How does optimistic detect a conflict?* A version/timestamp column; the update's `WHERE version = ?` matches zero rows if someone else changed it (JPA's `@Version`). *Downside of optimistic?* Wasted work under high contention (lots of failed retries), that is when pessimistic wins.

61. Explain indexes.

What they are testing: whether you understand the read/write trade-off, not just "indexes make queries fast."

Answer. An index is a **separate sorted data structure (usually a B-tree)** that lets the database **find rows without scanning the whole table**, turning an O(n) scan into an O(log n) lookup. The trade-off: indexes **speed up reads but slow down writes** (every insert/update/delete must also update the index) and **cost storage**. So you index columns you **filter, join, or sort on frequently**, and you do not index everything. A composite index's **column order matters** (it helps queries that filter on a left-most prefix).

Follow-ups. *Why not index every column?* Writes slow down and storage grows; unused indexes are pure cost. *Composite index order?* Leading column first, it only helps queries using the left-most prefix. *What is a covering index?* One that contains all columns a query needs, so it never touches the table.

62. Why is a query slow?

What they are testing: a diagnostic method, and whether you know to use `EXPLAIN`.

Answer. The first move is **`EXPLAIN` (the query plan)**, which tells you what the database is actually doing. The usual culprits it reveals: a **full table scan** (missing or unused index), a **bad join** (no index on the join column), the **N+1 problem** (many small queries instead of one), **`SELECT *`** pulling far more than needed, **no `LIMIT`** on a huge result, or **stale statistics** so the planner chooses badly. So I read the plan, find the scan or the expensive step, and fix the specific cause (usually an index). Guessing without `EXPLAIN` is the mistake.

Follow-ups. *What does EXPLAIN show?* The access path, indexes used, estimated rows, join strategy. *Query fast in dev, slow in prod?* Data volume and statistics differ; a full scan on 1000 rows is fine, on 10M it is not.

63. Explain transactions.

What they are testing: the concept behind ACID; keep it short since ACID (Q58) covers the guarantees.

Answer. A transaction is a **unit of work that is all-or-nothing**: a group of operations that either **all commit or all roll back**, leaving the database consistent. You `BEGIN`, do the work, and `COMMIT` (or `ROLLBACK` on failure). The classic example is a money transfer, debit one account and credit another must both happen or neither, or money vanishes. ACID (Q58) is the set of guarantees a transaction provides.

64. Explain the N+1 query problem.

What they are testing: a very common ORM performance bug; whether you recognise and fix it.

Answer. N+1 is when you run **1 query to fetch a list of parents, then N more queries, one per parent, to fetch each one's children**, so a list of 100 orders triggers 101 queries. It usually comes from **lazy-loading a relationship inside a loop** in an ORM like Hibernate. The fix is to fetch it in **one query**: a **join fetch** (`JOIN FETCH` in JPQL, or an entity graph), or **batch fetching** (`@BatchSize`) to at least turn N queries into a few. So you turn 101 queries into 1 (or a handful).

```
// N+1: this lazily loads items for each order, one query per order
orders.forEach(o -> process(o.getItems()));

// Fix: one query with a join fetch
// "SELECT o FROM Order o JOIN FETCH o.items"
```

Follow-ups. *How do you spot it?* Turn on SQL logging and see a burst of near-identical queries. *Join fetch downside?* It can multiply rows (a Cartesian-ish product) across multiple collections, use entity graphs or batch fetching then.

65. How would you optimise a database query?

What they are testing: a practical, ordered approach, not a single trick.

Answer. The way I work it: **`EXPLAIN` first** to see the real plan, then fix what it shows. The moves, roughly in order: **add or fix an index** on the filtered/joined columns (the biggest lever), **stop using `SELECT *`** (fetch only needed columns, enables covering indexes), **fix N+1** with a join fetch, **add a `LIMIT`** / paginate large results, **rewrite the query** if a subquery or function on an indexed column is defeating the index, and only then consider **denormalisation or caching** for genuinely hot read paths. So: measure with EXPLAIN, index, trim the columns, fix N+1, limit, and cache last. *(Adapt to your own examples.)*

Follow-ups. *Function on an indexed column?* `WHERE UPPER(name) = ?` cannot use a plain index on `name`, use a functional index or store normalised. *When to denormalise?* Only when a read path is hot and the join cost is proven, accepting the write complexity.